LoRA4VR — Training Log

A living record of experiments, findings, and open questions

What this is. A dated log of experiments run on the LoRA4VR self-play loop. Each entry records what was tried, what happened, and how to reproduce it. Entries are added as experiments complete. Reproduction references a git commit; the code at that commit is the exact code that produced the result.

System. `Qwen2.5-Coder-3B-Instruct` fine-tuned with GRPO/DAPO on a Google Cloud L4 GPU (24 GB). The loop generates `policy(state) -> action` Python code, scores it by running the policy in a sandboxed environment, and trains LoRA adapters on the reward signal.

2026-04-13 Environment design and first training runs

`d96f7cb` · *puck-on-tilted-plane env, runs 1–3*

Motivation

CartPole was the original seed task but it is trivially solved by a two-line bang-bang policy, giving a flat learning curve from iteration 0. Any standard RL benchmark (LunarLander, Pendulum, MountainCar) is heavily represented in the base model's pretraining data, so improvement from LoRA would be confounded with prior knowledge. We need an environment the model has to *read* rather than recall.

The puck environment

A puck moves in a bounded $[-6, 6]^2$ arena under a constant hidden tilt (t_x, t_y) and linear drag μ. The policy receives $\{p_x, p_y, v_x, v_y, g_x, g_y\}$ and must drive the puck to the goal within 120 steps. Hidden parameters — tilt, drag, thrust limit $a_{\max}$, initial velocity — are sampled per scenario from a seed and are not in the state dict. The tight thrust envelope ($a_{\max} = \|tilt\|_2 \times 1.05 + \epsilon$) ensures that a naive "thrust straight at the goal" policy fails because it cannot cancel tilt drift.

Reward: $-\Delta t \cdot \text{dist}(p, g)$ per step; $+50$ bonus on reaching the goal. Errored episodes score -200 per scenario (floor).

Baseline gap (200 seeds, hand-written policies):

- Naive aim: $\approx 26\%$ goal rate
- PD + tilt feedforward: $\approx 73\%$ goal rate

This gap is real headroom for LoRA to exploit.

Bugs encountered and fixed

Three bugs had to be resolved before any learning was possible.

Run 1 — OOM on L4. Default config ($G = 16$, batch 16, max completion 900 tokens) ran out of memory at the backward pass. Fix (PR #6): halved G and batch to 8, trimmed completion length to 600.

Run 2 — Reward sign flip. Training ran but degraded (fewer running samples each iteration). Errored samples scored 0.0 while samples that ran and drifted integrated large negative rewards (≈ -280). GRPO's normalized advantage placed the 0.0 errors *above* the mean, so the trainer rewarded broken code. Fix (PR #7): errored scenarios score -200, making broken code strictly the worst outcome.

Run 3 — Dead advantages. With LR $= 10^{-5}$ and only 1 training prompt (4 gradient steps per outer iteration), $\approx 75\%$ of batches were uniformly errored (advantage $= 0$, no gradient). On the remaining steps, weight deltas were $\sim 10^{-6}$ — indistinguishable from zero. The reward was identical to 7 decimal places at iteration 0 and iteration 9. Fix (PR #10): LR bumped $10\times$ to 10^{-4}; epochs raised to 20 (20 gradient steps per outer iteration on 1 prompt).

Reproduce

Commits: OOM fix `ef228fe`, reward floor `ef7e0c9`, LR/epoch fix `e9353d7`. Env source: `scripts/seed_tasks/puck_tilted_plane_env.py`.

2026-04-18 Run 5 probe: base model vs. 5 iterations of GRPO

`d5bc81c` · *puck_grpo_run_5, iters 0 and 5, $n = 20$ samples each*

Setup

Run 5 used the config at `e9353d7` (LR $= 10^{-4}$, $G = 8$, 20 epochs per outer iteration, DAPO loss, $\beta = 0$). It completed 5 outer iterations before the VM was stopped. Adapter checkpoints were recovered to `models/lora/puck_grpo_run_5/`.

To inspect what the model actually writes, we sample 20 policies from the base model (iter 0, no adapters) and from the trained model (iter 5), score each against 8 scenarios, and compare. The tool is `scripts/probe_generation.py`; it saves per-sample policy files, a summary CSV, and per-scenario reward breakdown to `data/probe/{run_name}_iter{N}_t{temp}/`.

Results

	Ran OK	Crashed	Mean reward (runners)	Best reward
Iter 0 (base)	13 / 20	7 / 20	−218.9	−92.4
Iter 5	20 / 20	0 / 20	−140.4	−140.4

Iter 0

The base model generates varied PD/PID controllers but crashes 35% of the time by attempting to read hidden env internals (`a_max`, `dt`, `steps`) from the state dict. The 13 successful policies span a reward range of $[-92, -373]$ with genuine variation in gain choices. The best policy uses asymmetric gains ($k_{px} = 2.0, k_{dx} = 1.0$; $k_{py} = 1.5, k_{dy} = 1.0$) and scores -92.4 over 8 scenarios.

Iter 5

Zero crashes. The error-reading habit is gone. However, all 20 samples at temperature 0.8 produce *exactly* −140.3516 (identical to 4 decimal places). Every policy is byte-for-byte the same: $k_p = 2.0$, $k_d = 1.5$. This is the **illustrative example embedded in the prompt template** (`configs/prompt_template.txt`). The model has memorized and reproduced it verbatim.

Interpretation

Training achieved two things and failed at a third.

Achieved. Error rate dropped from 35% to 0% — the model learned not to read hidden env attributes. Mean reward over runners improved by 35%.

Failed. The model collapsed to a single policy. With $\beta = 0$ (no KL penalty), one training prompt, and 20 gradient steps per outer iteration, GRPO found the prompt example as a high-reward fixed point and reinforced it into a delta distribution. By iter 5, temperature 0.8 sampling produces zero diversity. The best reachable policy is now −140.4, worse than the base model's best (−92.4).

This is reward hacking: the model optimized the training distribution (memorize the example) rather than the environment.

Hypotheses to test

1. **Remove or paraphrase the prompt example.** It is the attractor GRPO is collapsing to. A deliberately suboptimal or absent example removes the fixed point.
2. **More training prompts.** With 1 prompt, every outer iteration trains on the same input. Adding 4–8 tasks with varied dynamics forces generalization.
3. **Raise $\beta > 0$.** A non-zero KL coefficient prevents the policy from drifting too far from the base distribution and preserves diversity.
4. **Reduce epochs.** 20 gradient steps on 1 prompt per outer iteration is aggressive. 5–10 may preserve more exploration.
5. **Probe at every intermediate checkpoint.** Does collapse happen at iter 1 or gradually? One probe per checkpoint would pinpoint when diversity dies.

Open questions

- What does the W&B reward curve look like across the 5 iterations? (The run at `puck_grpo_run_5-orchestrator` has data but some post-resume steps may be missing due to a step-monotonicity issue.)
- What is the PD+feedforward ceiling expressed in probe-script reward units? This would establish what score a successful LoRA policy needs to beat.

Reproduce

At commit `d5bc81c`, with adapter checkpoints in `models/lora/puck_grpo_run_5/`:

```
scripts/probe_generation.py --iter 0 --run_name puck_grpo_run_5 --n 20
```

```
scripts/probe_generation.py --iter 5 --run_name puck_grpo_run_5 --n 20
```

Results land in `data/probe/puck_grpo_run_5_iter{000,005}_t0p80/`.